

vol-stream: decisions closed, then built

Five open items, all now decided with evidence — the research overturned **three of my own earlier recommendations**. The governing rule is **borrow by default**: we write the step semantics, the queue policy and the subscription protocol, because those are the differentiators. Everything else already exists in a library that does that one job better than we would.

LIBRARY CHANGES	none	OPEN ITEMS
0 of 5	HAND-WRITTEN FORMAT	none
DEPLOYMENT	Spack, from M0	

The finding that closed three decisions

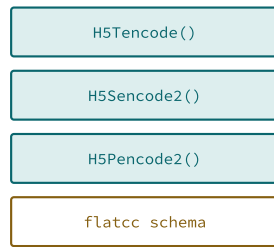
Verified against develop @ d006fb8d3a0.

HDF5 can serialize its own data model. `H5Tencode` encodes any datatype, `H5Sencode2` encodes a dataspace *including its selection*, and `H5Pencode2` encodes a property list — so chunking and the whole filter pipeline travel with the step.

This means the step manifest is not a new format. It is a thin envelope around blobs HDF5 produces and consumes, which makes the wire representation *exactly* as expressive as the data model — compound types, enums, opaque types, odd byte orders, arbitrary hyperslabs — with none of it hand-written. It also means a reader reconstructs types and selections with `H5Tdecode2` and `H5Sdecode` rather than interpreting our description of them.

The `H5Sencode2` selection encoding was widened to 64-bit in the 1.12 series, so large selections are not a constraint.

PRODUCED BY HDF5



ONE MANIFEST ENTRY

framed by flatcc — FlatBuffers schema



Nothing in a manifest entry is a hand-written format. HDF5 produces the datatype, the dataspace-with-selection, and the chunking-and-filter blobs; flatcc frames the record around them, contributing zero-copy in-place reads and real schema evolution — which is what my earlier hand-rolled envelope and its "reserved extension slot" were badly approximating. All we choose is which fields exist. Because the HDF5 blobs round-trip through the library's own decoders, a reader reconstructs the exact datatype and hyperslab the writer had, including cases nobody wrote a test for.

Borrow by default

The rule: we write only what is the differentiator. Everything else is somebody else's solved problem.

Applied honestly, this rule leaves a short list of things that are ours — which is the point. It also reorders the milestones: real transport arrives at M4 rather than last, because borrowing Mercury is cheaper than the MPI-only stopgap I had scheduled ahead of it.

The one boundary that holds: a *bounded library that does one job* is preferred over writing it; a *peer framework that owns the wire protocol* is not, because the protocol is where the differentiators live. That is the whole of the ADIOS2 argument and it does not extend to anything below.

JOB	BORROWED FROM	INSTEAD OF
datatype encoding	HDF5 <code>H5Tencode</code>	a type description format
space + selection encoding	HDF5 <code>H5Sencode2</code>	a hyperslab wire format
chunking + filter description	HDF5 <code>H5Pencode2</code>	a layout descriptor

JOB	BORROWED FROM	INSTEAD OF
M×N projection	HDF5 H5Sselect_project_intersection	redistribution algebra
manifest framing + evolution	flatcc (FlatBuffers in C)	hand-rolled envelope and a reserved extension slot
transport + RDMA	Mercury	socket and verbs plumbing
progress engine, threading	Argobots + Margo	a hand-written progress thread
rendezvous, group membership	SSG	contact files and join protocol
queue spill to node-local	BAKE	an on-disk spill format
precision / compression	ZFP · SZ via H5PL_TYPE_FILTER	a lossy codec
conformance testing	HDF5 test/API	a data-model test suite
build + deployment	Spack — mochi-margo is builtin	per-facility build instructions

What is genuinely ours

1. **Step semantics** — the physical/logical split, the restart-safe logical id set, and "state of group G as of step N."
2. **Queue policy** — the decision logic for Block, Discard and Spill. BAKE moves the bytes; choosing *when* is the differentiator.
3. **Subscription protocol** — what declaring interest means, how it is resolved against a step, and how precision is negotiated per subscriber.
4. **The HDF5-to-step mapping** — the VOL callbacks, capture, and replay. Nobody else can write this one.

Four items. If the list grows during implementation, that is the signal to go looking for a library rather than to start typing.

The five decisions

Question, evidence, decision. Where the evidence overturned an earlier recommendation of mine, that is marked.

1 · Is a step file-wide, or does each group get its own cadence?

EVIDENCE

openPMD hit this in production. Their ADIOS2 backend assumed one step equals one logical iteration in ascending order, and it *broke on restart from checkpoint*: resuming at iteration 500 after having written through 750 produces the sequence 0, 50, ... 750, 500, 550 — duplicate and non-monotone logical time on a monotone physical step counter.

Their fix was not a second step counter. It was an explicit `snapshot` attribute annotating which logical iteration(s) a physical step carries, with a fallback to linear progression when absent.

DECISION

A step is **file-scoped and physically monotone** — matching ADIOS2, which keeps atomicity and the queue simple. But the manifest carries an explicit, application-settable **set of logical step ids**, distinct from the physical counter.

This resolves the per-group-cadence question without a second mechanism: a group absent from step N simply retains its value from its last appearance, and readers ask for "the state of group G as of step N." A mesh written once per hundred field steps costs one manifest entry every hundred steps, not a re-send. It also fixes the restart overlap that ADIOS2 still has, which makes it a differentiator learned from someone else's production pain rather than invented.

2 · How does the step index appear in the dataspace?

EVIDENCE

openPMD standardized three encodings and ships all of them: *groupBased*, one group per iteration at `/data/<n>/`, which is what their HDF5 backend uses; *variableBased*, where a variable holds successive versions and no per-iteration group exists, which requires ADIOS2 and degrades to a single iteration on backends without it; and *fileBased*, one file per iteration.

Nothing in the ecosystem models the step as an extra array dimension. netCDF-4's record-dimension convention is the closest analogue and it applies to a variable's own shape, not to a container-wide transaction.

OVERTURNS MY EARLIER RECOMMENDATION

I recommended making the step an implicit slowest-varying dimension, so per-step `[ny][nx]` would present as `[nsteps][ny][nx]`. The evidence says don't: it conflates a transaction axis with a data axis, it breaks the moment a variable's per-step shape changes, and it has no precedent anyone reads.

DECISION

Two encodings, matching what already exists. On the wire, steps are **successive versions of the same named object** — openPMD's variable-based form, which is simply what a stream is. On replay and in the file engine, steps land **group-based** at `/step/<n>/`, because that is the portable HDF5 encoding openPMD already settled on and every existing tool can read it without knowing anything about streaming.

The `[nsteps][...]` aggregate view remains available, but as an *optional VDS overlay generated at replay* — a convenience projection, never the primary model.

3 · What is the subscription vocabulary, and what wire-format room does it need?

EVIDENCE

`H5Sencode2` encodes a dataspace together with its selection, so a hyperslab — including strided and irregular selections — is already a portable blob. `H5Pencode2` encodes a property list, so a filter pipeline with its parameters is too. Between them, three of the four things a subscription needs to say are expressible with no new format.

DECISION

The subscription is a list of (*object path*, `H5Sencode2` *blob*, optional `H5Pencode2` *blob*). Variables and subvolumes and stride come from the dataspace encoding; precision comes from the property-list encoding, which means per-subscriber precision reuses the existing filter pipeline instead of new compression code.

Only the predicate is genuinely new, and it does not ship until M7. No reserved slot is needed for it: because the manifest and subscription records are **FlatBuffers tables**, adding a field later is a compatible change that old readers ignore. Defining the schema in M2 is still the thing to get right, but the consequence of getting it slightly wrong is now an added field rather than a format break.

4 · Which parts of the data model are legal in v1?

EVIDENCE

`H5Dvlen_get_buf_size` and `H5Treclaim` let a connector size and release variable-length buffers, so it can deep-serialize an `hvl_t` into a step payload itself. And references are translatable: `H5Rget_obj_name`, `H5Rget_file_name` and `H5Rget_attr_name` convert an opaque `H5R_ref_t` into names, which `H5Rcreate_object` and `H5Rcreate_region` reconstruct on the far side — with the region's selection carried by `H5Sencode2`.

OVERTURNS MY EARLIER RECOMMENDATION

I said exclude variable-length data and references, citing VFD SWMR hitting the same wall because HDF5 stores VL data as metadata. That reasoning does not transfer. VFD SWMR's problem is *concurrent metadata in a shared file*; a connector that serializes its own payload never touches the global heap. The blocker I cited does not apply here.

DECISION

v1 covers datasets, attributes, groups, committed datatypes, and **variable-length data** via deep serialization. References are supported **by name translation** when the target is inside the stream.

The exclusion list shrinks to what is genuinely unrepresentable: references pointing outside the stream, which have no meaning at the far end and must be rejected at `end_step` with a clear error rather than silently dangling. Enforce all of this in the M2 test matrix so the boundary is tested, not assumed.

5 · What are the public calls named?

EVIDENCE

HDF5's house style, visible in `test/API/H5_api_async_test.c`, is to extend within the owning class namespace — `H5Dwrite_async`, `H5Fcreate_async` — or to add a new two-letter subsystem like `H5ES`. The Async VOL sets the out-of-tree precedent by shipping `H5F`-namespaced calls from a connector.

FIXES A COLLISION I INTRODUCED

My earlier draft used `H5SBegin_step`. `H5S` is the dataspace namespace — that name is actively misleading and would have shipped in a public header.

DECISION

`H5Fbegin_step()`, `H5Fend_step()`, `H5Fstep_status()`, `H5Fsubscribe()` — the file namespace, because a step is a file-scoped transaction, following the Async VOL precedent.

Registered operation strings are namespaced independently as `"vol-stream:begin_step"` so two connectors cannot collide in the optional-op registry, and a future connector that wants source compatibility can register the same strings.

Implementation design

The parts worth fixing before code, now that the decisions above constrain them.

Step manifest

```
// vol_stream.fbs – compiled by flatcc. Adding a field is a compatible change,
// so there is no reserved slot to guess the size of.
namespace vs;

enum Kind : ubyte { DsetCreate, DsetWrite, Attr, Group, Link }
enum Payload : ubyte { Raw, FilteredChunks }

table Entry {
  kind:Kind;
  path:string;
  type_enc:[ubyte];           // H5Tencode
  space_enc:[ubyte];         // H5Sencode2 – carries the selection
  dcpl_enc:[ubyte];          // H5Pencode2 – chunking + filter pipeline
  form:Payload;
  payload_off:ulong;
  payload_len:ulong;
  predicate:[ubyte];          // added in M7; absent in earlier writers
}

table Step {
  physical_step:ulong;        // monotone, connector-assigned
  logical_ids:[ulong];       // decision 1 – restart-safe
  wall_time_ns:ulong;         // supplied by caller, never generated
  hdf5_version:uint;          // which library produced the blobs above
  entries:[Entry];
  payload_bytes:ulong;
}
root_type Step;
```

The `Step` table is serialized after all entries are known, so a step remains a single forward-only append with no back-patching — the property that made the VOL the right layer in the first place. FlatBuffers' optional fields are why `predicate` can appear in M7 without a format break, and why `hdf5_version` costs nothing to carry from the first commit.

Filtered-chunk passthrough

`H5Dread_chunk2`, `H5Dwrite_chunk`, `H5Dchunk_iter` and `H5Dget_chunk_info` are all public, so when a dataset is chunked and filtered and a write covers whole chunks, the connector can move **already-compressed chunks** without decompressing and recompressing. That is the `FILTERED_CHUNKS` payload form.

This matters more than it looks. It cuts CPU on both ends, it shrinks the wire payload for free, and a chunk is the natural routing unit for subscription — pair it with `H5Sselect_intersect_block` and the writer decides which

chunks a subscriber needs without ever touching the data. Decide the fallback rule in M2: partial-chunk writes degrade to `RAW`.

Connector state machine

STATE	LEGAL OPERATIONS	NOTES
OPEN	create/open objects; <code>H5Fbegin_step</code> ; <code>H5Fsubscribe</code>	Writes outside a step are an error, not an implicit step — explicit beats surprising.
IN_STEP	dataset and attribute writes; further creates; <code>H5Fend_step</code>	Entries accumulate. Deferred writes queue against the connector's own request list.
COMMITTING	none from the application	Barrier over this step's requests, reference validation, manifest emit. Collective in parallel.
READING	<code>H5Fbegin_step</code> to advance; reads within the current step	Reader side. Advancing past an unconsumed step is governed by queue policy.

Registration and introspection

```
herr_t vs_init(hid_t vipl_id) {
    H5VLregister_opt_operation(H5VL_SUBCLS_FILE, "vol-stream:begin_step", &vs_op_begin);
    H5VLregister_opt_operation(H5VL_SUBCLS_FILE, "vol-stream:end_step", &vs_op_end);
    H5VLregister_opt_operation(H5VL_SUBCLS_FILE, "vol-stream:step_status", &vs_op_stat);
    H5VLregister_opt_operation(H5VL_SUBCLS_FILE, "vol-stream:subscribe", &vs_op_sub);
    return 0;
}

/* opt_query lets applications and tools discover this without a capability flag. */
herr_t vs_opt_query(void *obj, H5VL_subclass_t cls, int op, uint64_t *flags) {
    if (op == vs_op_begin || op == vs_op_end)
        *flags = H5VL_OPT_QUERY_SUPPORTED | H5VL_OPT_QUERY_COLLECTIVE
            | H5VL_OPT_QUERY_MODIFY_METADATA;
    ...
}
```

`H5VL_OPT_QUERY_COLLECTIVE` is the standard way to state that step boundaries must be called by every rank — so the parallel contract is discoverable through an existing mechanism rather than only documented.

Threading

Argobots and Margo own the progress engine from M4, not from the end. Margo exists precisely to drive Mercury from user-level threads, so the alternative was writing a progress thread by hand for three milestones and then deleting it. The connector keeps one lock over its own step and queue state, which is our concurrency to reason about; network progress is not.

v1 still does not advertise `H5VL_CAP_FLAG_THREADSAFE`. Borrowing a good threading library is not the same as promising the connector is safe under concurrent application calls, and claiming that before the queue is stable buys nothing.

Milestones

Sizes are relative — S is days, M is weeks, L is a quarter-ish for one developer. Every gate is runnable.

M0

Skeleton, CI, and the regression net SIZE S

Start from `H5VLPassthru.c`, a complete pass-through connector already in the tree. Register as `vol-stream`, build as a shared plugin, wire CI across every HDF5 version you intend to support. No streaming logic yet — this exists purely to make later milestones measurable.

EXIT GATE

`test/API` driven via `HDF5_VOL_CONNECTOR=vol-stream` scores identically to native. Any nonzero delta is a skeleton bug.

M1

Step API surface, as no-ops SIZE S

Register the four namespaced operations, ship `H5VLstream.h` with `H5Fbegin_step`, `H5Fend_step`, `H5Fstep_status` and `H5Fsubscribe` over `H5VLfile_optional_op`, implement `opt_query` including the collective flag, and pin the `H5VL_VERSION` check. Decision 5 is settled, so this is mechanical.

EXIT GATE

An application brackets writes in `H5Fbegin_step` / `H5Fend_step` and produces a byte-identical file to one written without them. `H5VLquery_optional` reports each op correctly, including collectivity.

M2

Manifest, capture, and the replay invariant SIZE M

The milestone that carries the weight an RFC would have. Define the flatcc schema and fill it from `H5Tencode`, `H5Sencode2` and `H5Pencode2`; implement the logical-id set from decision 1; land steps group-based per decision 2; implement VL deep-serialization and reference-by-name from decision 4.

Then stand up the replay invariant: the same unmodified writer through the connector and through native HDF5 must produce indistinguishable files.

EXIT GATE

`h5diff` clean across compound and enum types, byte-order mismatch, committed datatypes, VL data, in-stream references, attributes, nested groups, chunked and contiguous layouts, and partial hyperslab writes. Out-of-stream references fail at `end_step` with a specific error. Filtered-chunk passthrough round-trips, and partial-chunk writes demonstrably fall back to `RAW`.

M3

Decoupled reader SIZE M

A separate reader process opens the stream, iterates steps, and reads with hyperslabs unrelated to how the writer decomposed its writes. Single-process both sides, to prove the selection machinery before MPI is in the picture. Implement "state of group G as of step N" from decision 1, and the optional VDS aggregate overlay from decision 2.

EXIT GATE

Selections straddling multiple writer calls return correct data. A group written every hundredth step resolves correctly at every intermediate step. A restart-overlap sequence — logical ids 0...750 then 500... — is read back in correct logical order.

M4

Mercury transport, Margo progress, deferred I/O

SIZE M

Real network transport arrives here rather than at the end. `tr_mercury` with Margo driving progress on Argobots threads, and `H5VL_request_class_t` implemented so writes queue and resolve at `end_step` — the barrier being a wait over the connector's own request list.

Pulling this forward is the direct consequence of borrow-by-default: the MPI-only stopgap I had scheduled ahead of it existed only to postpone a dependency, and would have been written, tested and then largely superseded.

EXIT GATE

`H5_api_async_test.c` passes. Writer and reader in separate processes over Mercury on both `na+sm` and `ofi+tcp`, replay invariant intact with deferral on.

M5

Rendezvous and late joiners

SIZE S

SSG for group membership, which is what SST's `RegistrationMethod`, `RendezvousReaderCount` and late-joiner discovery amount to. No shared filesystem assumption, and no contact-file protocol of ours to maintain.

EXIT GATE

A writer starts with no readers and proceeds; a reader attaches at step 500 and gets a coherent view; a reader leaving mid-stream does not stall the writer. All without a shared filesystem between them.

M6

Parallel writer and real M×N

SIZE L

Collective `begin_step`/`end_step` over the writer communicator, per-rank selections aggregated into the manifest, reader requests resolved with `H5Sselect_project_intersection`. Aggregation topology borrowed from the Subfiling VFD's I/O concentrators.

EXIT GATE

Byte-exact data with coprime rank counts — 7 writers to 3 readers, 64 to 5. Replay invariant holds in parallel.

M7

Queue policy and BAKE spill

SIZE M

The queue with `Block`, `Discard` and `Spill`, plus reserve slots and latest-step-only for monitoring readers. BAKE moves bytes to node-local storage; the policy logic deciding *when* is ours, because that is the differentiator over ADIOS2's two-way choice.

EXIT GATE

Under a deliberately slow reader: `Block` stalls the writer and loses nothing, `Discard` drops only whole steps, `Spill` does neither and the lagging reader catches up from local storage.

M8

Subscription protocol

SIZE L

The differentiator, and the reason the protocol had to be ours. Readers declare interest as *(path, `H5Sencode2`, `H5Pencode2`)* triples per decision 3; the writer marshals only what is subscribed, routing at chunk granularity with `H5Sselect_intersect_block`. Precision negotiation is the existing filter pipeline. Mercury's RPC direction, already in place since M4, is the back-channel.

EXIT GATE

Two subscribers on one stream at different precisions from a single `end_step`. Measured wire bytes scale with subscribed volume, not total step volume — the number that proves the thesis.

M9

Tools, bindings, and the long tail

SIZE M

`h5stream` promoted from test scaffolding to a real tool: list steps, tail a live stream, reorganize into a static file. `h5py` bindings for the step and subscription calls. Document the tool-compatibility matrix honestly, `h5dump` included.

Then the roadmap items: predicate pushdown, which needs per-chunk statistics that `H5D_chunk_rec_t` does not carry; onion-backed addressable step history; and an ADIOS2 interop bridge through its Plugin Engine.

EXIT GATE

A viewer written entirely in Python follows a live C or Fortran writer, subscribing to a subvolume, with no user-written C glue.

CI matrix

Version skew is the failure mode with precedent — the ADIOS2 connector broke when `H5VL_VERSION` moved. It is 3 today.

AXIS	VALUES	WHY
HDF5 version	develop · latest release · oldest supported	The documented failure mode, and <code>H5VL_VERSION</code> will move again.
Mercury NA plugin	<code>na+sm</code> · <code>ofi+tcp</code> · <code>ofi+verbs</code>	Where transport bugs actually live. Shared-memory and TCP run on any CI box.
Spack env	pinned lockfile · latest deps	The pinned build is reproducible; the floating one detects upstream breakage early.
MPI	MPICH · OpenMPI	Intercommunicator and dynamic-process behaviour differ in practice.
rank shapes	$1 \rightarrow 1 \cdot 7 \rightarrow 3 \cdot 64 \rightarrow 5$	Coprime counts are where $M \times N$ projection bugs surface.
encode round-trip	cross-endian pair	The manifest leans entirely on HDF5's encoders; prove them across byte order.

AXIS	VALUES	WHY
sanitizers	ASan · UBSan · TSan	A progress thread plus a queue is the classic place for races.

Residual risks

What is left after closing the five decisions.

The manifest inherits HDF5's encoding compatibility rules

Leaning on `H5Tencode` and `H5Sencode2` is the right trade, but it couples the wire format to those encodings. `H5Sencode2` already changed once to widen selections to 64-bit. Record the producing library version in the step header from the first commit, and put a cross-version encode round-trip in CI, so a future change is a detected incompatibility rather than a corrupt read.

Existing tools on a live stream

`h5dump` assumes random access and will fail unhelpfully. Without a capability flag there is no clean detection path, so the mitigation is `opt_query`, a tested error-message matrix, and shipping `h5stream` early. If this becomes the top adoption complaint, that is the evidence for proposing the flag upstream — driven by reports, not speculation.

VL and reference support widen the M2 surface

Including them is correct on the evidence, but it makes M2 the largest early milestone and deep-copying VL data has real cost. If M2 slips, ship VL and references behind a property defaulting to off rather than cutting the replay-invariant matrix — the matrix is what protects everything after.

The dependency risk is quality, not quantity

Counting dependencies is the wrong metric — an unmaintained one-function library is worse than five well-supported ones. The criteria to apply before adding anything: is it actively maintained, is it already deployed at the facilities we target, is it Spack-installable, is its scope bounded, and does it stay out of the wire protocol. Mercury, Argobots, Margo and flatcc pass all five; `mochi-margo` and Mercury are builtin Spack packages, and SSG and BAKE come from the Mochi Spack namespace with a published buildcache and facility platform configs.

What still needs enforcing is a **pinned Spack environment checked into the repo from M0**, so a build is reproducible and an upstream change surfaces as a CI failure on the floating environment rather than as a user's broken install.

Sources

1. [openPMD-api #949](#) — mapping ADIOS2 steps to openPMD iterations
2. [openPMD](#) — iteration encodings
3. [ADIOS2](#) — supported engines
4. [ADIOS2](#) — SST engine parameters
5. [Mercury](#) — RPC and bulk transfer
6. [Mochi](#) — composable HPC data services
7. [Mochi Spack packages](#)
8. [Mochi Spack buildcache](#)
9. [flatcc](#) — FlatBuffers for C
10. [HDF5 Async I/O VOL connector](#)
11. [HDF5 Cache VOL connector](#)
12. [VFD SWMR User's Guide](#)
13. [HDF5 filter plugins](#)